Course Outline: Gestionando tablas relacionadas con SQL

Title: Gestionando tablas relacionadas con SQL **Learnpack:** + Gestionando tablas relacionadas con SQL **Prerequisite:** Consultando bases de datos con SQL en PostgreSQL (Fundamentos de SQL) **Target Audience:** Beginner developers learning relational database design and advanced SQL operations **Total Lessons:** 11 **Format:** Mixed (45% hands-on)

Course Description

This course teaches students how to design and manage relationships between database tables using SQL. Students will learn to create entity-relationship diagrams, implement different types of relationships through foreign keys, and write complex queries that join multiple tables. The course emphasizes normalization principles and best practices for maintaining data integrity in relational databases.

By the end of this course, students will confidently design relational database schemas, implement foreign key relationships, and write sophisticated JOIN queries to retrieve data across multiple related tables.

Course Philosophy

This course emphasizes:

- **Relational thinking**: Understanding how real-world entities map to database relationships
- **Data integrity**: Using foreign keys and normalization to maintain consistent, reliable data
- **Query efficiency**: Writing JOIN operations that retrieve exactly the data needed
- **Practical application**: Building relationships that solve real business problems

---

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - Understanding Database Relationships	3
02 - Types of Relationships	2
03 - Advanced Relationship Patterns	2
04 - Querying Related Data	2
05 - Normalization and Cascade Operations	2
06 - Outro	1
Total	**11**

Section 00: Welcome

00.0 Welcome to Relational Database Management 📖

Learning Objectives:

- Understand the core problem this course solves: managing related data efficiently
- Preview the journey from isolated tables to sophisticated relational schemas

- Recognize the business value of properly designed database relationships
- Connect this course to the broader database management curriculum

Content Outline:

- The limitation of isolated tables: why single-table designs break down
- Preview of course capabilities: from simple foreign keys to complex JOIN operations
- Real-world scenarios where relationships are essential (e-commerce, social media, inventory)
- Course section overview and learning progression

Transitions: This welcome lesson sets the stage for understanding database relationships. Next, we'll dive into the fundamental concepts of how tables relate to each other through foreign keys.

Key Principles:

- Related data requires relational design patterns
- Foreign keys are the foundation of data integrity
- Proper relationships eliminate data duplication and inconsistency
- Complex business logic maps naturally to relational structures

Examples:

- E-commerce scenario: customers, orders, and products that must connect reliably
- Blog platform: users, posts, and comments that reference each other
- Library system: books, authors, and borrowers with multiple relationship types

---

Section 01: Understanding Database Relationships

01.0 What Are Database Relationships and Entity-Relationship Diagrams 📖

Learning Objectives:

- Define database relationships and explain their purpose in relational design
- Create entity-relationship (E/R) diagrams that map real-world scenarios to database structure
- Identify entities and relationships in business scenarios
- Understand how E/R diagrams guide database implementation

Content Outline:

- Database relationships defined: connections between tables that reflect real-world associations
- Entity-relationship diagrams: visual tools for designing relational schemas
- Entities vs. relationships: distinguishing things from connections
- Mapping real-world scenarios to database structure
- E/R diagram notation and conventions

Transitions: Now that we understand how to visualize relationships, we'll learn how to implement them technically using foreign keys, the mechanism that creates connections between tables.

Key Principles:

- Relationships reflect real-world associations between entities
- E/R diagrams bridge business requirements and technical implementation
- Visual modeling prevents design mistakes before coding begins
- Every relationship must serve a clear business purpose

Examples:

- University system: Students enroll in Courses (many-to-many relationship)
- Company hierarchy: Employees work in Departments (many-to-one relationship)

- Customer orders: Each Order belongs to one Customer (one-to-many relationship)

Anti-patterns woven in:

- **Storing related data as text fields**: Instead of creating a separate Categories table, storing "Electronics, Gadgets, Tech" as a text field in Products table. This violates normalization, makes querying difficult, and leads to data inconsistency.
- **Avoiding foreign keys entirely**: Creating tables without establishing formal relationships, relying on application code to maintain integrity. This leads to orphaned records and data corruption.

01.1 Foreign Keys and Table Connections 📖

Learning Objectives:

- Understand how foreign keys create and enforce relationships between tables
- Apply the naming convention for foreign key columns
- Explain referential integrity and its importance for data consistency
- Distinguish between foreign keys and primary keys in relational design

Content Outline:

- Foreign keys defined: columns that reference primary keys in other tables
- Best practice naming convention: `table_name_id` format
- Referential integrity: ensuring foreign keys always point to existing records
- Creating foreign key constraints in SQL
- The relationship between primary keys and foreign keys

Transitions: With foreign keys established as our connection mechanism, we're ready to implement our first table relationships in practice, creating the foundational connections that will support our more complex relationship patterns.

Key Principles:

- Foreign keys are references, not copies of data
- Referential integrity prevents orphaned records
- Consistent naming conventions improve code readability
- Constraints should be enforced at the database level, not just in application code

Examples:

- `orders` table with `customer_id` foreign key referencing `customers.id`
- `comments` table with `post_id` foreign key referencing `posts.id`
- `employees` table with `department_id` foreign key referencing `departments.id`

Anti-patterns woven in:

- **Inconsistent foreign key naming**: Using various formats like `custId`, `customer`, `cust_pk` instead of the standard `customer_id`. This creates confusion and makes relationships hard to follow.
- **Not enforcing foreign key constraints**: Creating columns that logically should be foreign keys but without database constraints, allowing invalid references to be inserted.

01.2 Building Your First Table Relationships 💻

Challenge Prompt: Create a database schema with customers and orders tables, establishing a proper foreign key relationship where each order belongs to exactly one customer.

Solution Code:

```
 -- Create customers table first (referenced table must exist)
CREATE TABLE customers (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
```

```
    email VARCHAR(100) UNIQUE NOT NULL
);

-- Create orders table with foreign key
CREATE TABLE orders (
    id SERIAL PRIMARY KEY,
    order_date DATE NOT NULL DEFAULT CURRENT_DATE,
    total_amount DECIMAL(10,2) NOT NULL,
    customer_id INTEGER NOT NULL,
    FOREIGN KEY (customer_id) REFERENCES customers(id)
);

-- Insert sample data
INSERT INTO customers (name, email) VALUES
    ('Alice Johnson', 'alice@email.com'),
    ('Bob Smith', 'bob@email.com');

INSERT INTO orders (total_amount, customer_id) VALUES
    (99.99, 1),
    (149.50, 1),
    (75.25, 2);

-- Verify the relationship works
SELECT c.name, o.total_amount, o.order_date
FROM customers c
JOIN orders o ON c.id = o.customer_id;
```

Challenge Code:

```
 -- Create customers table first (referenced table must exist)
CREATE TABLE customers (
    -- Your customer table structure here
);

-- Create orders table with foreign key
CREATE TABLE orders (
    -- Your orders table structure with foreign key here
);

-- Insert sample data to test the relationship
-- Your INSERT statements here

-- Write a query to verify the relationship works
-- Your JOIN query here
```

Section 02: Types of Relationships

02.0 One-to-One, One-to-Many, and Many-to-Many Relationships 📖

Learning Objectives:

- Distinguish between one-to-one, one-to-many, and many-to-many relationship types
- Identify which relationship type fits specific business scenarios
- Understand the implementation differences between relationship types
- Recognize when to choose each relationship pattern

Content Outline:

- One-to-one relationships: when each record connects to exactly one other record
- One-to-many relationships: the most common pattern in relational databases
- Many-to-many relationships: when records can connect to multiple other records
- Implementation strategies for each relationship type
- Business scenarios that require each pattern

Transitions: Now that we understand the three core relationship types conceptually, we'll implement each one using SQL, creating the foreign key structures and constraints that make these relationships work in practice.

Key Principles:

- One-to-many is the most common and straightforward relationship type
- Many-to-many relationships require intermediate tables (pivot tables)
- One-to-one relationships are rare and often indicate design issues
- The business requirements determine the relationship type, not technical convenience

Examples:

- One-to-one: User and UserProfile (each user has exactly one profile)
- One-to-many: Customer and Orders (each customer can have many orders)
- Many-to-many: Students and Courses (each student takes many courses, each course has many students)

Anti-patterns woven in:

- **Force-fitting one-to-many when many-to-many is needed**: Creating multiple foreign key columns like `student_course1_id`, `student_course2_id` instead of using a proper junction table. This limits flexibility and violates normalization.
- **Using one-to-one when data should be in the same table**: Creating separate tables for data that logically belongs together, leading to unnecessary JOINs and complexity.

02.1 Creating Different Relationship Types with SQL 💻

Challenge Prompt: Implement all three relationship types in a school database: one-to-one between users and profiles, one-to-many between teachers and courses, and many-to-many between students and courses.

Solution Code:

```
 -- One-to-one: users and profiles
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL
);

CREATE TABLE profiles (
    id SERIAL PRIMARY KEY,
    bio TEXT,
    avatar_url VARCHAR(200),
    user_id INTEGER UNIQUE NOT NULL,
    FOREIGN KEY (user_id) REFERENCES users(id)
);

-- One-to-many: teachers and courses
CREATE TABLE teachers (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL
);

CREATE TABLE courses (
    id SERIAL PRIMARY KEY,
```

```
    title VARCHAR(100) NOT NULL,
    teacher_id INTEGER NOT NULL,
    FOREIGN KEY (teacher_id) REFERENCES teachers(id)
);

-- Many-to-many: students and courses (requires pivot table)
CREATE TABLE students (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL
);

CREATE TABLE enrollments (
    student_id INTEGER,
    course_id INTEGER,
    enrollment_date DATE DEFAULT CURRENT_DATE,
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES students(id),
    FOREIGN KEY (course_id) REFERENCES courses(id)
);
```

Challenge Code:

```
 -- One-to-one: users and profiles
-- Your one-to-one implementation here

-- One-to-many: teachers and courses
-- Your one-to-many implementation here

-- Many-to-many: students and courses
-- Your many-to-many implementation here (don't forget the pivot table!)
```

Section 03: Advanced Relationship Patterns

03.0 Pivot Tables for Many-to-Many Relationships 📖

Learning Objectives:

- Understand why pivot tables are necessary for many-to-many relationships
- Design effective pivot table structures with appropriate primary keys
- Add meaningful attributes to pivot tables beyond just foreign keys
- Implement composite primary keys for junction tables

Content Outline:

- The pivot table concept: junction tables that resolve many-to-many relationships
- Composite primary keys: using multiple columns as the primary key
- Adding attributes to pivot tables: enrollment dates, quantities, status fields
- Naming conventions for pivot tables
- Query patterns for accessing data through pivot tables

Transitions: With pivot tables mastered as our solution for complex relationships, we'll now implement a comprehensive example that demonstrates how multiple relationship types work together in a real-world database schema.

Key Principles:

- Pivot tables eliminate the need for duplicate foreign key columns
- Composite primary keys prevent duplicate relationships

- Pivot tables can store relationship-specific data
- Junction tables should be named to reflect their purpose

Examples:

- `order_items` pivot table connecting orders and products with quantity and price
- `project_assignments` connecting employees and projects with role and start date
- `course_enrollments` connecting students and courses with grade and enrollment date

Anti-patterns woven in:

- **Using text fields to store multiple values**: Instead of a pivot table, storing "Course1, Course2, Course3" in a text field. This makes querying impossible and violates first normal form.
- **Creating separate tables for each relationship instance**: Making `student_course_1`, `student_course_2` tables instead of using a single pivot table with foreign keys.

03.1 Implementing Complex Relationships 💻

Challenge Prompt: Build a complete e-commerce database with products, categories, orders, and customers, implementing a many-to-many relationship between products and categories using a proper pivot table.

Solution Code:

```
 -- Core entities
CREATE TABLE customers (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL
);

CREATE TABLE categories (
    id SERIAL PRIMARY KEY,
    name VARCHAR(50) NOT NULL
);

CREATE TABLE products (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    price DECIMAL(10,2) NOT NULL
);

-- Pivot table for products and categories (many-to-many)
CREATE TABLE product_categories (
    product_id INTEGER,
    category_id INTEGER,
    PRIMARY KEY (product_id, category_id),
    FOREIGN KEY (product_id) REFERENCES products(id),
    FOREIGN KEY (category_id) REFERENCES categories(id)
);

-- Orders with line items
CREATE TABLE orders (
    id SERIAL PRIMARY KEY,
    customer_id INTEGER NOT NULL,
    order_date DATE DEFAULT CURRENT_DATE,
    FOREIGN KEY (customer_id) REFERENCES customers(id)
);

CREATE TABLE order_items (
```

```
    order_id INTEGER,
    product_id INTEGER,
    quantity INTEGER NOT NULL,
    unit_price DECIMAL(10,2) NOT NULL,
    PRIMARY KEY (order_id, product_id),
    FOREIGN KEY (order_id) REFERENCES orders(id),
    FOREIGN KEY (product_id) REFERENCES products(id)
);
```

Challenge Code:

```
 -- Create your e-commerce schema here
-- Remember: customers, products, categories, orders
-- Don't forget the pivot tables for many-to-many relationships

-- Your table definitions here
```

Section 04: Querying Related Data

04.0 JOIN Operations: INNER, LEFT, RIGHT, and OUTER 📖

Learning Objectives:

- Understand the purpose and behavior of different JOIN types
- Choose the appropriate JOIN type for specific data retrieval needs
- Predict the result set size and content for each JOIN operation
- Write efficient queries that combine data from multiple related tables

Content Outline:

- INNER JOIN: returning only matching records from both tables
- LEFT JOIN: preserving all records from the left table
- RIGHT JOIN: preserving all records from the right table
- OUTER JOIN: combining LEFT and RIGHT JOIN behaviors
- When to use each JOIN type based on business requirements
- Performance considerations for different JOIN operations

Transitions: With JOIN theory established, we'll now practice writing complex queries that use multiple JOIN operations to extract meaningful business insights from our related tables.

Key Principles:

- INNER JOIN is the most restrictive and commonly used
- LEFT JOIN preserves the completeness of the primary entity
- RIGHT JOIN is rarely used (LEFT JOIN with reversed tables is preferred)
- OUTER JOIN provides the complete picture of both tables

Examples:

- INNER JOIN: customers with orders (only customers who have placed orders)
- LEFT JOIN: all customers with their orders (including customers with no orders)
- RIGHT JOIN: all orders with customer information (rare use case)
- OUTER JOIN: complete view of customers and orders relationship

Anti-patterns woven in:

- **Using INNER JOIN when LEFT JOIN is needed**: Missing important records because of overly restrictive joins. For example, using INNER JOIN to get "all customers and their orders" will exclude customers who haven't placed orders yet.
- **Overusing RIGHT JOIN instead of reordering LEFT JOIN**: RIGHT JOIN makes queries harder to read and understand. Always prefer LEFT JOIN with proper table ordering.

04.1 Mastering JOIN Queries for Related Tables 💻

Challenge Prompt: Write a comprehensive query that joins customers, orders, and order items to show complete order information, including handling customers who haven't placed orders yet.

Solution Code:

```
 -- Sample data setup
INSERT INTO customers (name, email) VALUES
    ('Alice Johnson', 'alice@email.com'),
    ('Bob Smith', 'bob@email.com'),
    ('Carol White', 'carol@email.com');

INSERT INTO products (name, price) VALUES
    ('Laptop', 999.99),
    ('Mouse', 29.99),
    ('Keyboard', 79.99);

INSERT INTO orders (customer_id) VALUES (1), (1), (2);
INSERT INTO order_items (order_id, product_id, quantity, unit_price) VALUES
    (1, 1, 1, 999.99),
    (1, 2, 2, 29.99),
    (2, 3, 1, 79.99),
    (3, 2, 1, 29.99);

-- Comprehensive JOIN query
SELECT
    c.name AS customer_name,
    c.email,
    o.id AS order_id,
    o.order_date,
    p.name AS product_name,
    oi.quantity,
    oi.unit_price,
    (oi.quantity * oi.unit_price) AS line_total
FROM customers c
LEFT JOIN orders o ON c.id = o.customer_id
LEFT JOIN order_items oi ON o.id = oi.order_id
LEFT JOIN products p ON oi.product_id = p.id
ORDER BY c.name, o.id, p.name;
```

Challenge Code:

```
 -- Write your comprehensive JOIN query here
-- Requirements:
-- 1. Show all customers (even those without orders)
-- 2. Include order details when they exist
-- 3. Show product information for each order item
-- 4. Calculate line totals

-- Your query here
```

Section 05: Database Normalization and Cascade Operations

05.0 Normalization Principles and Cascade Delete Operations 📖

Learning Objectives:

- Understand database normalization and its benefits for data integrity
- Explain how normalization eliminates redundancy and update anomalies
- Describe cascade delete operations and their use cases
- Recognize when cascade operations are appropriate vs. risky

Content Outline:

- Database normalization defined: organizing data to reduce redundancy
- Benefits of normalization: eliminating update anomalies, saving storage, ensuring consistency
- First, second, and third normal forms (brief overview)
- Cascade delete operations: automatically removing dependent records
- When to use CASCADE vs. RESTRICT on foreign key constraints
- Risks and safety considerations with cascade operations

Transitions: We've covered the complete spectrum of relational database management, from basic relationships to complex queries. Our final assessment will test your mastery of all these concepts in realistic scenarios.

Key Principles:

- Normalization prevents data inconsistency and reduces storage waste
- Each piece of information should be stored in exactly one place
- Cascade operations should be used carefully with proper understanding of their impact
- Database design decisions have long-term consequences for data integrity

Examples:

- Normalized design: separate customers, addresses, and orders tables vs. denormalized single table
- Cascade delete: removing a customer automatically deletes their orders
- Restrict delete: preventing category deletion when products still reference it

Anti-patterns woven in:

- **Over-denormalization for perceived performance**: Storing customer names in every order record "to avoid JOINs." This creates update anomalies when customer names change and wastes storage.
- **Unsafe cascade deletes**: Setting up CASCADE deletes on critical relationships without understanding the impact. For example, deleting a product category and accidentally removing all products in that category.

05.1 Relational Database Mastery Assessment 🧠

Learning Objectives:

- Demonstrate understanding of relationship types and their implementations
- Apply JOIN operations to solve complex data retrieval problems
- Evaluate database design decisions for normalization and integrity
- Analyze the impact of foreign key constraints and cascade operations

Assessment Questions:

1. **Multiple Choice**: You have a blog where users can write posts, and each post can have multiple tags, while each tag can be used on multiple posts. What type of relationship exists between posts and tags?
    - a) One-to-one
    - b) One-to-many
    - c) Many-to-many ✓

- d) No relationship needed

2. **Multiple Choice**: When would you use a LEFT JOIN instead of an INNER JOIN?
   - a) When you want to improve query performance
   - b) When you want to include records from the left table even if they have no matches in the right table ✓
   - c) When you want to exclude records that don't have matches
   - d) When you want to join more than two tables

3. **Multiple Choice**: What is the recommended naming convention for foreign key columns?
   - a) Just use "id" for all foreign keys
   - b) Use the referenced table name + "_id" ✓
   - c) Use "fk_" prefix + column name
   - d) Use random descriptive names

4. **Multiple Choice**: In a properly normalized database, customer information appears in:
   - a) Every table that needs customer data
   - b) Both the customers table and orders table
   - c) Only the customers table, referenced by foreign keys ✓
   - d) A central lookup table

5. **Multiple Choice**: What happens when you try to insert an order with a customer_id that doesn't exist in the customers table?
   - a) The order is inserted with a null customer_id
   - b) A new customer record is automatically created
   - c) The database returns a foreign key constraint violation error ✓
   - d) The customer_id is set to a default value

6. **Multiple Choice**: When implementing a many-to-many relationship between students and courses, the pivot table should:
   - a) Have only the two foreign key columns
   - b) Use a composite primary key of the two foreign keys ✓
   - c) Have its own separate auto-incrementing primary key
   - d) Not have a primary key at all

7. **Multiple Choice**: CASCADE DELETE on a foreign key constraint means:
   - a) The delete operation will be faster
   - b) Deleting a referenced record will also delete all records that reference it ✓
   - c) The delete operation will be rolled back if there are references
   - d) Only the foreign key values will be set to NULL

Score Interpretation:

- 6-7 correct: Excellent understanding of relational database concepts
- 4-5 correct: Good grasp with some areas to review
- 2-3 correct: Basic understanding, recommend reviewing JOIN operations and relationship types
- 0-1 correct: Fundamental concepts need reinforcement

Section 06: Outro

06.0 Your Relational Database Journey Forward

Content Outline:

- Course recap: You've mastered the fundamentals of relational database design, from simple foreign keys to complex JOIN operations and normalization principles

- What you can now build: Design complete database schemas for real applications, implement proper relationships between entities, and write sophisticated queries that extract meaningful insights from related data
- Connection to bigger picture: These relational concepts form the foundation for advanced database topics like indexing, query optimization, and distributed database systems
- Next steps in your database journey: Advanced SQL operations (window functions, CTEs), database performance tuning, NoSQL databases for specific use cases, and database administration
- Resources for continued learning: PostgreSQL documentation for advanced features, database design pattern books, online communities for SQL practitioners
- Encouragement: Relational database skills are fundamental to most software applications — you now have the foundation to build robust, scalable data layers for any project

Note: This is conclusion only - no theory, exercises, or assessment.
